PyQGIS - Python no QGIS

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

15 de março de 2026

Arquitetura do QGIS

QGIS é estruturado em camadas (layers de software):

- **Core** → Modelo de dados geoespacial
- **GUI** → Interface gráfica (Qt)
- **Analysis** → Algoritmos e Processing
- **Providers** → OGR, GDAL, PostGIS, etc.

PyQGIS é o binding Python da API C++ do QGIS.

QgsApplication e Inicialização

Dentro do QGIS:

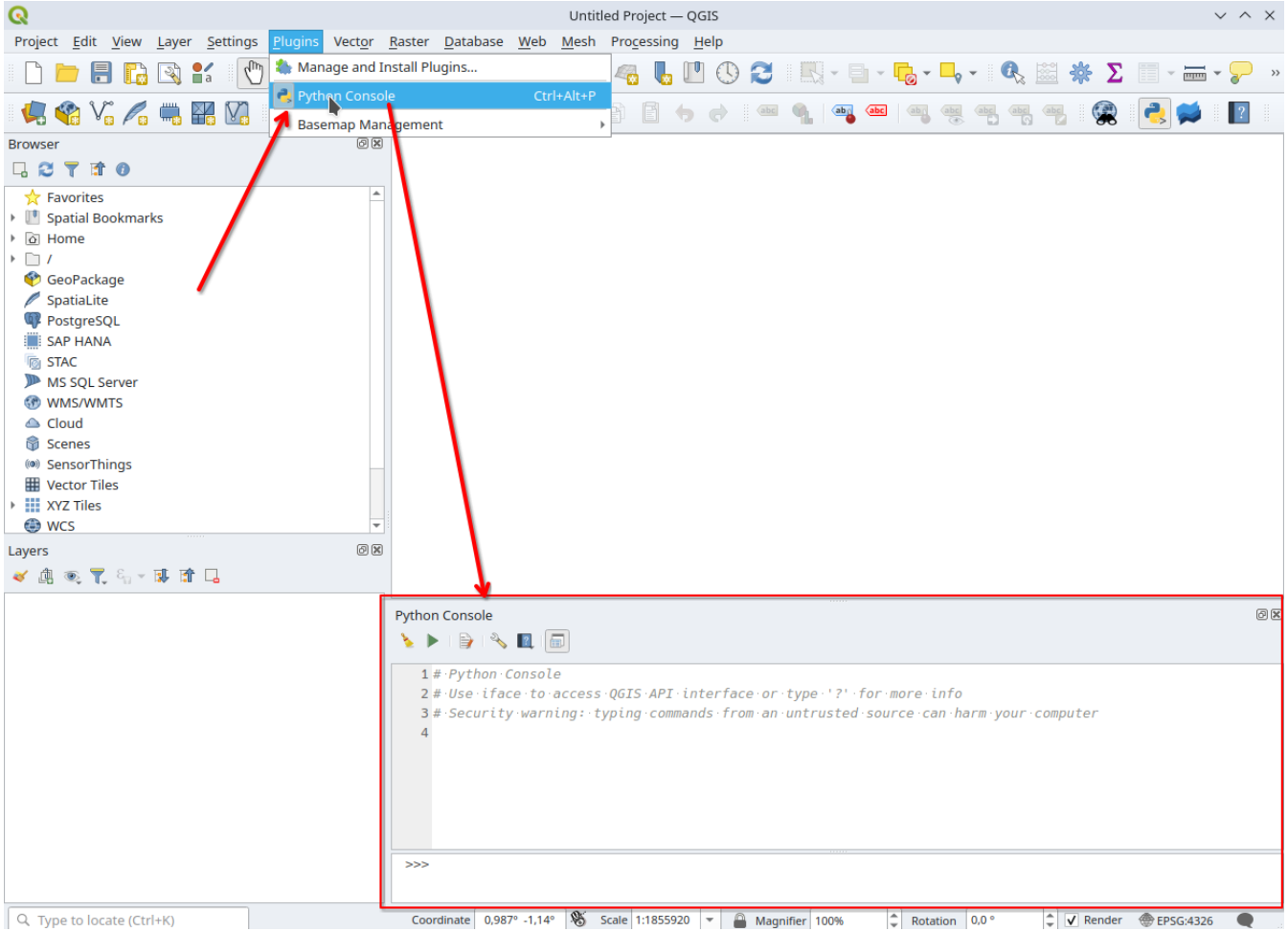
```
# Já inicializado automaticamente
layer = iface.activeLayer()
```

Script standalone:

```
from qgis.core import QgsApplication

QgsApplication.setPrefixPath("/usr", True)
app = QgsApplication([], False)
app.initQgis()

# código aqui

app.exitQgis()
```

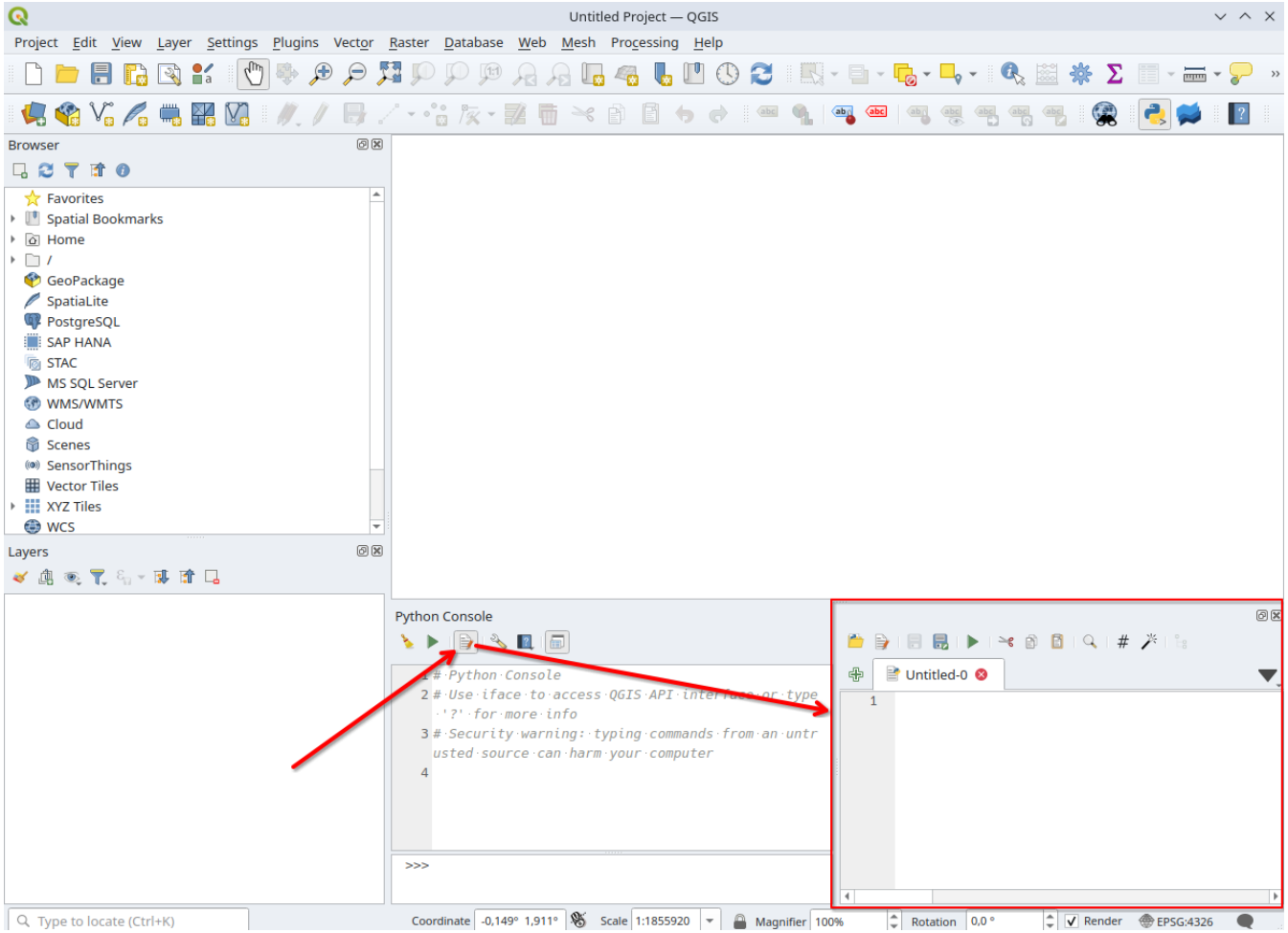
Scripts Python no QGIS

Terminal Python O terminal Python do QGIS permite executar comandos de forma interativa.



Scripts Python no QGIS

Editor de scripts Python O Editor de scripts Python do QGIS permite editar e executar scripts inteiros.



Manipulação da estrutura do Projeto no QGIS

QgsProject

- Representa o projeto aberto (.qgz)
- Gerencia camadas
- Singleton

Obter a instância (global) do projeto

```
from qgis.core import QgsProject
project = QgsProject.instance()
```

Permite:

- Adicionar camadas
- Remover camadas
- Acessar CRS do projeto

PyQGIS — QgsProject.instance()

QgsProject.instance() fornece acesso ao **projeto QGIS atualmente aberto**. É o ponto central para acessar **camadas, CRS, caminhos e estado do projeto**.

Gerenciamento de camadas

```
project.mapLayers()

project.mapLayersByName("rios")

project.addMapLayer(layer)

project.removeMapLayer(layer.id())
```

Informações do projeto

```
project.fileName()

project.homePath()

project.crs()

project.setCrs(crs)
```

Exemplo: listar camadas

```
for layer in project.mapLayers().values():
    print(layer.name())
```

Providers

Providers são responsáveis pelo acesso aos dados

- OGR → Shapefile, GeoPackage
- GDAL → Raster
- PostGIS → Banco espacial

Ao criar uma camada:

```
QgsVectorLayer(path, name, "ogr")
```

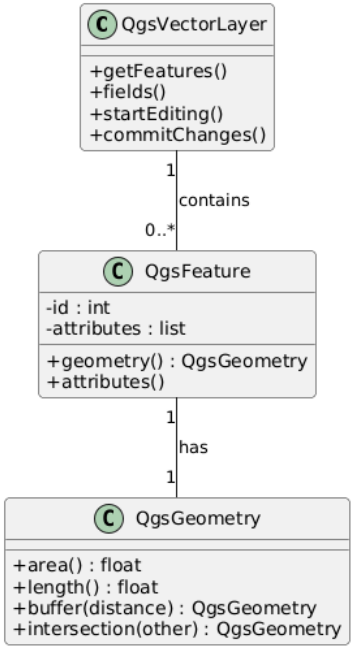
"ogr" é o provider.

Modelo de Dados Vetorial

Principais classes:

- **QgsVectorLayer**
- **QgsFeature**
- **QgsGeometry**
- **QgsField / QgsFields**

Diagrama de classes



Relação conceitual:

$$\text{Layer} \rightarrow \text{Features} \rightarrow \text{Geometry} + \text{Attributes}$$

QgsVectorLayer: diferentes fontes

A classe `QgsVectorLayer` permite acessar dados vetoriais de diversos formatos.

```
from qgis.core import QgsVectorLayer, QgsProject
```

Shapefile

```
layer = QgsVectorLayer(
"/dados/rios.shp",
"Rios",
"ogr"
)
```

GeoPackage

```
layer = QgsVectorLayer(
"/dados/brasil.gpkg|layername=estados",
"Estados",
"ogr"
)
```

PostGIS

```
uri = "dbname='gis' host=localhost \
port=5432 user='postgres' \
password='123' \
table=\"public\".\"municipios\" (geom)"

layer = QgsVectorLayer(uri,
"Municipios",
"postgres")
```

Adicionar ao projeto

```
QgsProject.instance().addMapLayer(layer)
```

Iterando Feições

```
layer = iface.activeLayer()

for feat in layer.getFeatures():
    geom = feat.geometry()
    attrs = feat.attributes()

    print(geom.area(), attrs)
```

Observação:

- Iteração é lazy (generator)
- Geometria e atributos são acessados separadamente

Edição de Camadas

```
layer.startEditing()

for feat in layer.getFeatures():
    feat["area"] = feat.geometry().area()
    layer.updateFeature(feat)

layer.commitChanges()
```

Regra importante:

- Sempre iniciar e finalizar modo de edição

Geometria (QgsGeometry)

Operações comuns:

- `area()`
- `length()`
- `buffer()`
- `intersection()`
- `difference()`

Conversões:

- WKT
- WKB
- GeoJSON

Geometria (QgsGeometry)

Operações geométricas

```
from qgis.core import QgsGeometry

g1 = QgsGeometry.fromWkt("POINT(0 0)")
g2 = QgsGeometry.fromWkt("POINT(1 1)")


g_line = QgsGeometry.fromWkt("LINESTRING(0 0, 1 1, 2 1)")
print(g_line.length()) # comprimento

g_poly = QgsGeometry.fromWkt(
"POLYGON((0 0,2 0,2 2,0 2,0 0))"
)
print(g_poly.area()) # area

# buffer
buffer_geom = g1.buffer(10, 8)

# operações espaciais
inter = g_poly.intersection(buffer_geom)
diff = g_poly.difference(buffer_geom)
```

Geometria (QgsGeometry): Criação

A classe `QgsGeometry` pode ser criada a partir de objetos `QgsPointXY`, listas de pontos ou anéis de polígonos.

Exemplos

```
from qgis.core import QgsGeometry, QgsPointXY

# POINT
p = QgsPointXY(-43.2, -22.9)
geom_point = QgsGeometry.fromPointXY(p)

# LINESTRING
line = [ QgsPointXY(0,0), QgsPointXY(1,1), QgsPointXY(2,1) ]
geom_line = QgsGeometry.fromPolylineXY(linha)

# POLYGON
ring = [[ QgsPointXY(0,0), QgsPointXY(2,0), QgsPointXY(2,2),
          QgsPointXY(0,2), QgsPointXY(0,0)
]]
geom_poly = QgsGeometry.fromPolygonXY(ring)
```

Geometria (QgsGeometry): Acesso às coordenadas

Convertendo QgsGeometry para estruturas Python

```
from qgis.core import QgsGeometry, QgsWkbTypes

# acessar coordenadas
p = g1.asPoint()
line = g_line.asPolyline()
poly = g_poly.asPolygon()

print(p)
print(line)
print(poly)
```

Comparando o tipo da geometria

```
geom = g_line
if geom.wkbType() == QgsWkbTypes.LineString:
    print("É uma linha")
if geom.wkbType() == QgsWkbTypes.Point:
    print("É um ponto")
if geom.wkbType() == QgsWkbTypes.Polygon:
    print("É um polígono")
```

Geometria (QgsGeometry)

Também existem as versões que usam outras geometrias (e não QgsPointXY)

```
QgsGeometry.fromPolyline(
    [QgsPoint(1, 1), QgsPoint(2, 2)]
)
```

Conversões de formato

```
# WKT
wkt = g_poly.asWkt()

# WKB
wkb = g_poly.asWkb()

# GeoJSON
json = g_poly.asJson()
```

Geometria (QgsGeometry)

É possível iterar sobre as partes de geometrias mais complexas:

Para Multipontos

```
geom = QgsGeometry.fromWkt( 'MultiPoint( 0 0, 1 1, 2 2)' )
for part in geom.parts():
    print(part.asWkt())
```

Para Linestrings

```
geom = QgsGeometry.fromWkt( 'LineString( 0 0, 10 10 )' )
for part in geom.parts():
    print(part.asWkt())
```

Exemplos de WKT (Well-Known Text)

WKT é um formato textual padrão para representar geometrias.

Geometrias simples

```
POINT(10 20)

LINESTRING(0 0, 1 1, 2 1)

POLYGON(
    (0 0, 4 0, 4 3, 0 3, 0 0)
)
```

Geometrias múltiplas

```
MULTIPOINT(
    (10 20),
    (30 40)
)

MULTILINESTRING(
    (0 0, 1 1, 2 1),
    (2 2, 3 3)
)

MULTIPOLYGON(
    ((0 0,4 0,4 3,0 3,0 0)),
    ((5 5,7 5,7 7,5 7,5 5))
)
```

Exemplo: Camada de memória com um ponto

Podemos criar camadas temporárias diretamente em memória usando `QgsVectorLayer`.

```
from qgis.core import (
QgsVectorLayer, QgsFeature, QgsGeometry, QgsPointXY, QgsProject
)

# criar camada de memória
layer = QgsVectorLayer("Point?crs=EPSG:4326", "Pontos", "memory")
# acessar provedor de dados
provider = layer.dataProvider()
# criar feição
feat = QgsFeature()
geom = QgsGeometry.fromPointXY(QgsPointXY(-43.2, -22.9))
feat.setGeometry(geom)
# adicionar feição na camada
provider.addFeature(feat)
# atualizar camada
layer.updateExtents()
# adicionar ao projeto
QgsProject.instance().addMapLayer(layer)
```

Exemplo: Camada de memória com linha geodésica

```
from qgis.core import QgsDistanceArea
new_york = (40.661, -73.944) #lat lon
london = (51.5072, 0.1276) #lat lon
# Remember the order is X,Y
point1 = QgsPointXY(new_york[1], new_york[0])
point2 = QgsPointXY(london[1], london[0])
d = QgsDistanceArea()
d.setEllipsoid('WGS84')
# Create a geodesic line with vertices every 100km
vertices = d.geodesicLine(point1, point2, 100000)
# Create a polyline from the vertices
# The method returns 1 geometry unless the line crosses antimeridien
geodesic_line = QgsGeometry.fromPolylineXY(vertices[0])
# Create a line layer to display the route
vlayer = QgsVectorLayer('LineString?crs=EPSG:4326', 'route', 'memory')
provider = vlayer.dataProvider()
f = QgsFeature()
f.setGeometry(geodesic_line)
provider.addFeature(f)
vlayer.updateExtents()
QgsProject.instance().addMapLayer(vlayer)
```

CRS e Transformações

Classes importantes:

- **QgsCoordinateReferenceSystem**
- **QgsCoordinateTransform**

Transformação típica:

- CRS origem
- CRS destino
- Contexto do projeto

Essencial para operações métricas corretas.

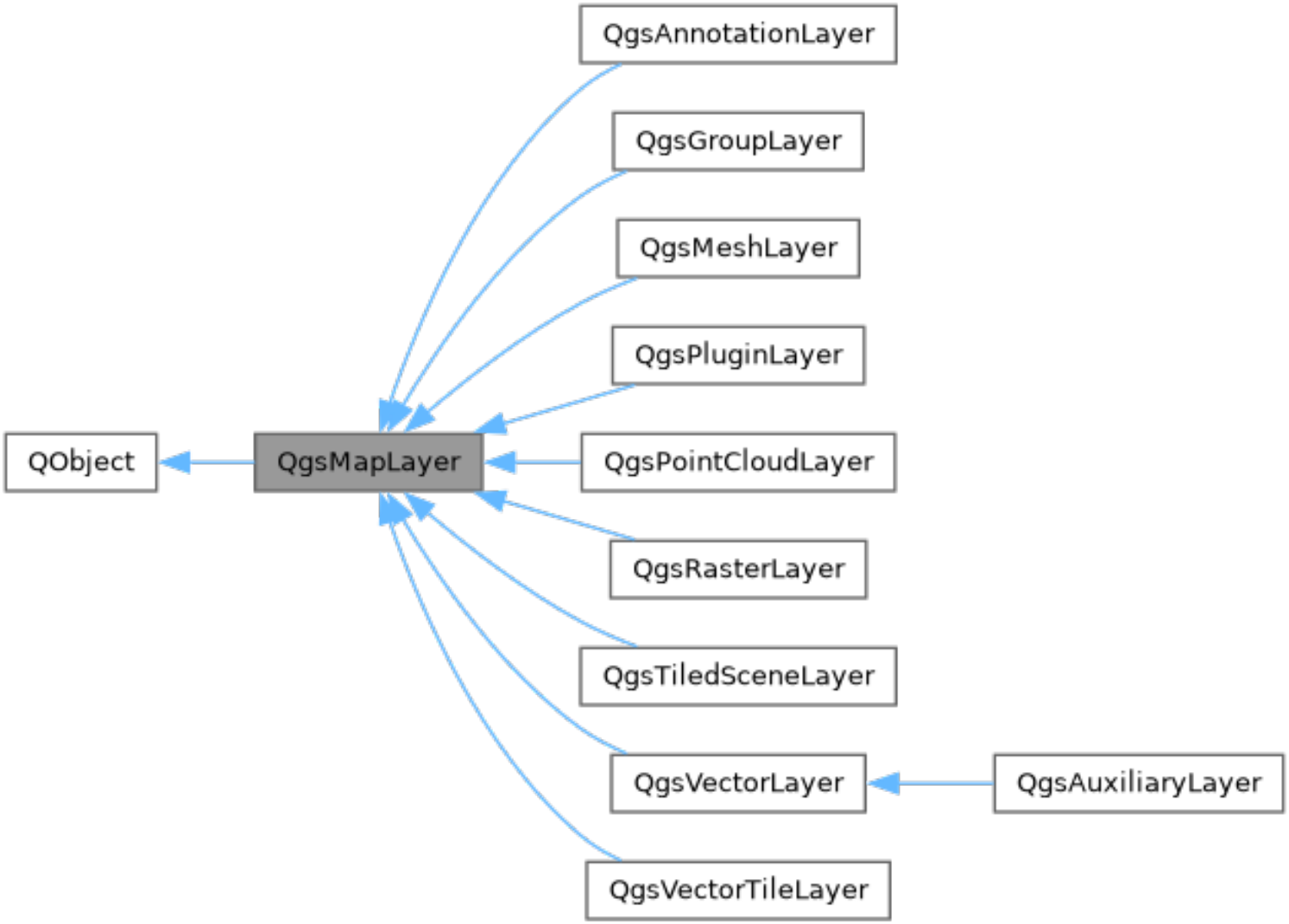
Transformação de CRS (PyQGIS)

O PyQGIS permite transformar geometrias entre sistemas de referência.

```
from qgis.core import (
QgsCoordinateReferenceSystem,
QgsCoordinateTransform,
QgsProject,
QgsGeometry
)
# geometria em WGS84
geom = QgsGeometry.fromWkt("POINT(-43 -22)")
# CRS origem e destino
src = QgsCoordinateReferenceSystem("EPSG:4326")
dst = QgsCoordinateReferenceSystem("EPSG:3857")
# criar transformador
transform = QgsCoordinateTransform(src, dst, QgsProject.instance())
# transformar geometria
geom.transform(transform)
print(geom.asWkt())
```

Exemplo: conversão de coordenadas geográficas (lat/lon) para Web Mercator.

Tipos de camada no QGIS



https://api.qgis.org/api/classQgsMapLayer.html

Automação e Scripts Standalone

Objetivo:
Executar rotinas de geoprocessamento fora da interface gráfica do QGIS.

Casos típicos:

- Processamento em lote
- Integração com pipelines ETL (extract, transform, load)
- Execução em servidor
- Agendamento (cron / task scheduler)

Comparação: Standalone vs Plugin

	Plugin	Standalone
Interface gráfica	Sim	Não
Execução agendada	Limitado	Sim
Escalabilidade	Média	Alta
Uso em servidor	Difícil	Ideal

Arquitetura: GUI vs Standalone

Dentro do QGIS:

- iface disponível
- Projeto já carregado
- Ambiente inicializado

Standalone:

- Inicializar QApplication
- Inicializar QgsApplication
- Configurar prefixPath

Estrutura Mínima de Script Standalone

```python
from qgis.core import *
from qgis.PyQt.QtWidgets import QApplication
import sys

QgsApplication.setPrefixPath(
    "/usr", True
)

app = QApplication(sys.argv)
qgs = QgsApplication([], False)
qgs.initQgis()

# --- código aqui ---

qgs.exitQgis()
```

Componentes Críticos

QgsApplication:

- Inicializa provedores
- Carrega drivers GDAL/OGR
- Gerencia ambiente QGIS

setPrefixPath():

- Define onde o QGIS está instalado
- Essencial para localizar recursos internos

Carregando Camada Vetorial

```
layer = QgsVectorLayer(
    "/dados/limites.shp",
    "limites",
    "ogr"
)

if not layer.isValid():
    raise Exception("Camada inválida")
```

Importante: Sem projeto QGIS carregado, tudo deve ser instanciado manualmente.

Executando Processing Standalone

```
import processing
from processing.core.Processing import Processing

Processing.initialize()

result = processing.run("native:buffer", {
    "INPUT": layer,
    "DISTANCE": 100,
    "OUTPUT": "/dados/buffer.shp"
})
```

Observação: Processing precisa ser explicitamente inicializado.

Automação em Lote

Padrão típico:

- Listar arquivos com glob/os
- Iterar camadas
- Executar algoritmo
- Salvar saída

Aplicações:

- Processamento municipal
- Geração de buffers por estado
- Reprojeção em massa

Integração com CLI

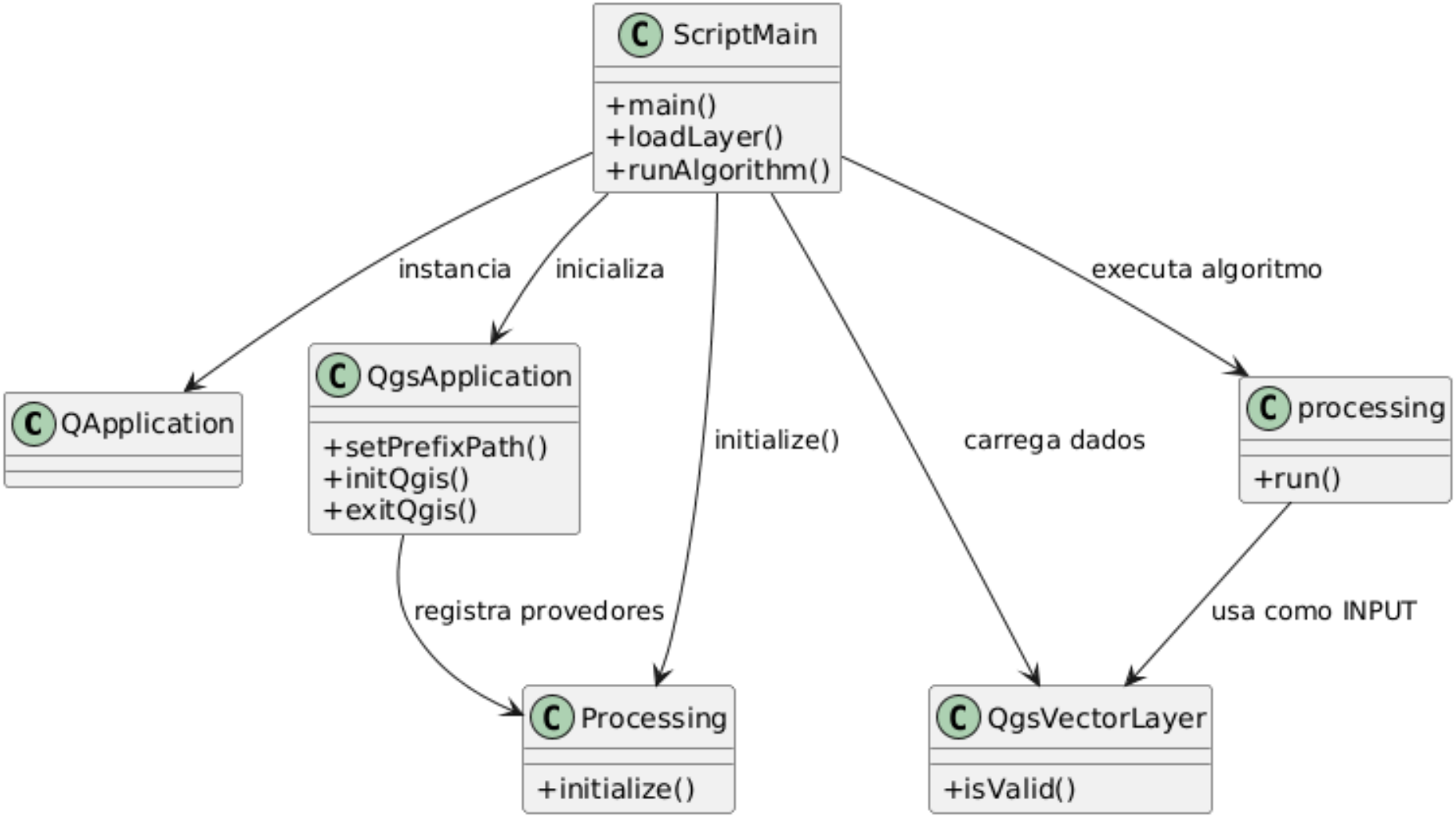
Script pode receber argumentos:

- argparse
- variáveis de ambiente
- arquivos de configuração (YAML/JSON)

Permite:

- Execução parametrizada
- Integração com Airflow
- Pipelines CI/CD

Boas Práticas

- Encapsular lógica em funções
- Evitar código global
- Validar camadas antes de processar
- Usar logs ao invés de print
- Garantir qgs.exitQgis()

Arquitetura Standalone — PyQGIS